

4장 데이터 계약 및 DB 설계

4.1 데이터 계약(Data Contract) 정의 원칙

계층 간 데이터 전달의 형식·책임을 명시한 약속. 세 가지 원칙:

원칙	내용
계층별 검증 책임	FastAPI=형식(Pydantic), DRF=참조무결성(Serializer), DB=제약(Constraint)
raw / 파생 분리	센서 raw 값과 서버 계산값(위험도)을 분리 저장
단일 진실 공급원	임계치는 DB Threshold 1곳 → FastAPI·DRF 동일 참조

4.2 메시지/이벤트 스키마 규격 (필드·타입·단위)

도메인	핵심 필드	타입·단위
가스	co~voc (9종), lel	float, ppm/%
전력	channel(1~16), data_type, value	int, watt/A/V
위치	worker_id, x, y, node_id	int, float(≥0)
알람	risk_level, alarm_type, event_id	str enum

- 공통: device_id(str), measured_at(datetime ISO8601)
 - 검증: Pydantic Field(ge/le) 범위 제약 (예: o2 0~100, x/y ≥ 0)
- 전력 채널키: watt/current/voltage=int키, onoff=str키
 - 미등록 센서: Redis Set 1차 검증 → 404 차단

4.3 센서 입력 데이터 표준화 규약

[가스] status 무시 → 서버 재계산 / lel은 raw_payload에만
[전력] slaveNN → 채널번호(1~16) / -1 → None(comm_failure)
[위치] username → worker_id FK 매핑 (시작 시 1회 조회)

4.4 논리 ERD 및 테이블 설계

GasSensor	——< GasData	(9종 값 + 위험도)
PowerDevice	——< PowerData	(채널별 시계열)
	└——< PowerEvent	(ON/OFF 스냅샷)
Worker	——< Event ——< AlarmRecord	
Threshold		(임계치 단일출처)
MLModel	——< MLAnomalyResult	(AI 결과)
DataRetentionPolicy		(보존정책)
도메인: monitoring (측정)		
alerts (알람)		
ml (AI)		
operations (운영)		
facilities (설비 · 임계치)		

<데이터 모델 관계 요약도>

설명: 센서·설비·작업자·이벤트·알람·AI 결과·운영정책 간의 주요 엔티티 관계를 텍스트 기반 ERD 형식으로 정리한 도메인 모델 구조이다.



4.5 주요 엔티티별 제약조건

엔티티	핵심 컬럼	제약조건	의도
GasData	9종 가스값·9종 위험도 ·max_risk_level·raw_payload	gas_sensor FK = PROTECT / 가 스값 위험도 null=True / lel은 컬 럼 없이 raw_payload에만	센서 삭제 시 측정이력 보존, 결 측 허용
PowerData	power_device·channel(1~16)·data _type·measured_at	power_device FK = PROTECT / UniqueConstraint (device·channel·data_type·meas ured_at)	동일 시점 채널 중복 저장 차단
Event	facility·status·source_sensor·sourc e_power_device·source_geofence	facility FK = PROTECT / 발생원 3 종 = SET_NULL, 정확히 1개만 NOT NULL / status 상태머신	소속 보존 + 발생원 단일화 + 생 명주기 관리
AlarmRecord	facility·algorithm_source·source·s ensor·ml_anomaly_result	facility FK = PROTECT / 나머지 6 FK = SET_NULL / save-delete 차단(불변) / algorithm_source × source 직교 2축	판정 순간을 불변 기록으로 영구 보존

<엔티티별 컬럼·제약·의도>

4.6 시계열·이력 데이터 저장 전략 (보존·파티셔닝)

DataRetentionPolicy 기반 자동 정리 (Celery 매일 실행)

카테고리	raw 보존	
gas_raw / power_raw	30일	
gas_anomaly / power_agg	30일	
position_hist	30일	

- 배경: SQLite 시절 raw 무제한 누적 → 12GB 폭증 → 30일 retention 도입
- 효과: 12GB → 160MB (임시적 감소)

4.7 인덱싱 및 조회 성능 설계

엔티티	근거
GasData 4개	gas_data.py:189-205
PowerData 5개+unique	power_data.py:84-105
Event 5개(부분3)	event.py:144-167
AlarmRecord 3개	alarm_record.py:224-232

<인덱스 정의 근거>

설계 원칙 — 모든 인덱스를 (필터 컬럼 + 시간 역순) 복합 구조로 만들었습니다.

- 센서·이벤트는 “조건으로 거른 뒤 최신부터” 보는 패턴이라, 필터와 정렬을 한 인덱스로 묶어 정렬 비용
을 없앴습니다.

엔티티별 인덱스

- GasData — 센서별·전체·위험도별·이상탐지별 4개 인덱스로, “어느 센서가 언제 얼마나 위험했는지”를
모두 커버합니다.
- PowerData — 장치 아래 채널(1~16) 구조라, (장치, 채널, 시간)까지 좁혀 채널 단위 최신 조회를 최적
화했습니다.
- Event — 대시보드의 “활성 이벤트 필터·최신 정렬”을 인덱스로 받칩니다. 특히 발생원 3종은 부분 인덱
스로, NULL 인 행은 빠고 값 있는 행만 인덱싱해 공간까지 아꼈습니다.
- AlarmRecord — 이벤트별·센서별·시설유형별 3개 인덱스로 알람 묶음·이력·집계 조회를 지원합니다.

<인덱스 설계 원칙>

엔티티	인덱스 (필드)	조회 용도
GasData	(gas_sensor, -measured_at)	센서별 최신 측정 조회
	(-measured_at)	전체 최신순 정렬
	(max_risk_level, -measured_at)	위험도별 필터
PowerData	(is_anomaly, -measured_at)	이상탐지 결과 조회
	(power_device, channel, - measured_at)	장치·채널별 최신 조회
	(power_device, channel, data_type, -measured_at)	측정종류별 조회
Event	(-measured_at)	전체 최신순 정렬
	(risk_level, -measured_at)	위험도별 필터
	(is_anomaly, -measured_at)	이상탐지 결과 조회
AlarmRecord	(facility, status, risk_level)	활성 이벤트 필터 (조치필요/조치중 탭)
	(facility, -created_at)	최신 이벤트 정렬 (이벤트 현황 패널)
	(source_sensor, status) ※부분	발생원별 활성 이벤트 (병합 로직)
	(source_power_device, status) ※부 분	//
	(source_geofence, status) ※부분	//
AlarmRecord	(event, -created_at)	이벤트에 묶인 알람 조회
	(sensor, -created_at)	센서별 알람 이력
	(facility, alarm_type, -created_at)	시설·유형별 알람 조회

<전체 인덱스 및 조회 용도>

4.8 스키마 버전 관리 및 호환성 정책

항목	정책
마이그레이션	Django migration 순차 관리, 인덱스명 직접 명명(PG 63자 제한 대응)
DB 전환	SQLite → PostgreSQL 16 (2026-05-22), fixture historical 패턴 통일
하위 호환	Pydantic Literal 확장(anomaly_type 4→6종), 옛 필드 유지
설정 분리	settings dev/prod 분기 (DJANGO_DEBUG 기준)

<마이그레이션·DB전환·하위호환 정책>

